

SISTEMAS MULTIMÉDIA I

ENGENHARIA DE REQUISITOS

Helder Rodrigo Pinto

OBJETIVOS

Instituto Superior de Tecnologias Avançadas do Porto

- Metodologias de Desenvolvimento
- Engenharia de Requisitos
- User Stories
- Diagrama de Casos de Uso
- BPMN

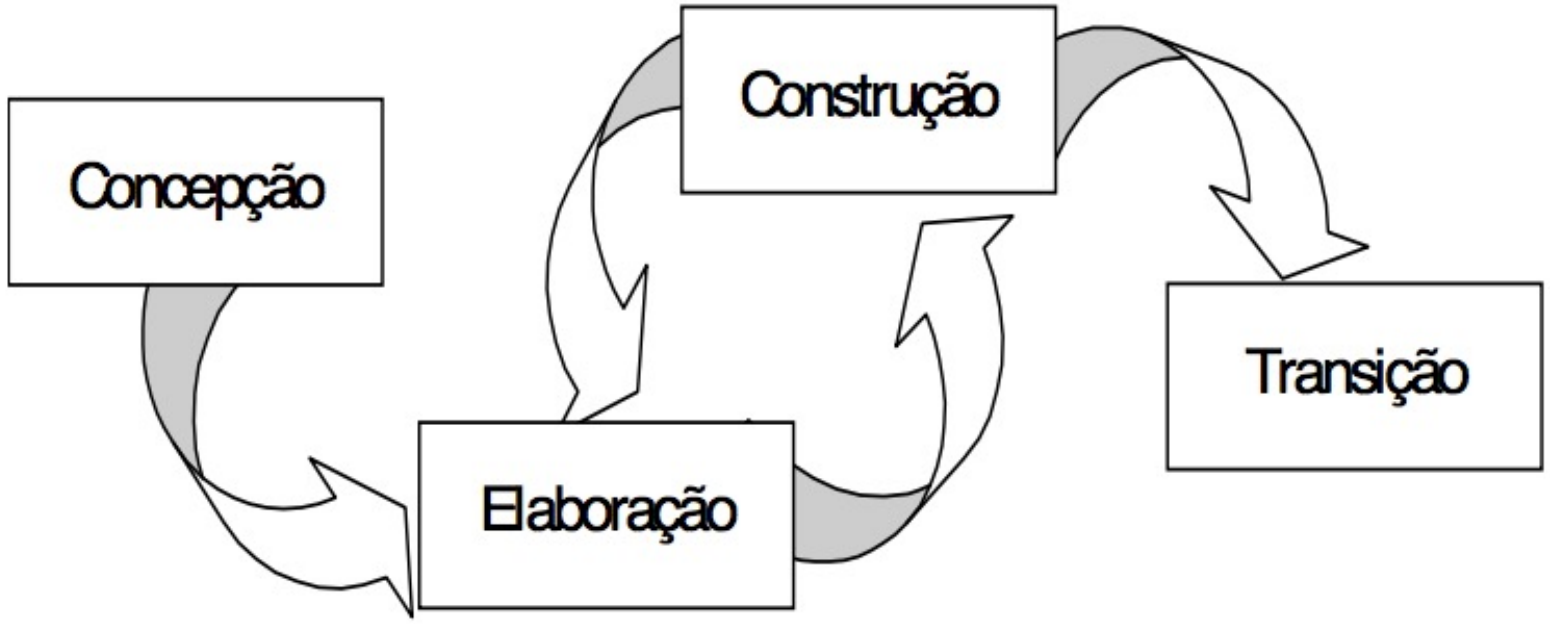
OBJETIVOS

- Metodologias de Desenvolvimento
- Engenharia de Requisitos
- User Stories
- Diagrama de Casos de Uso
- BPMN

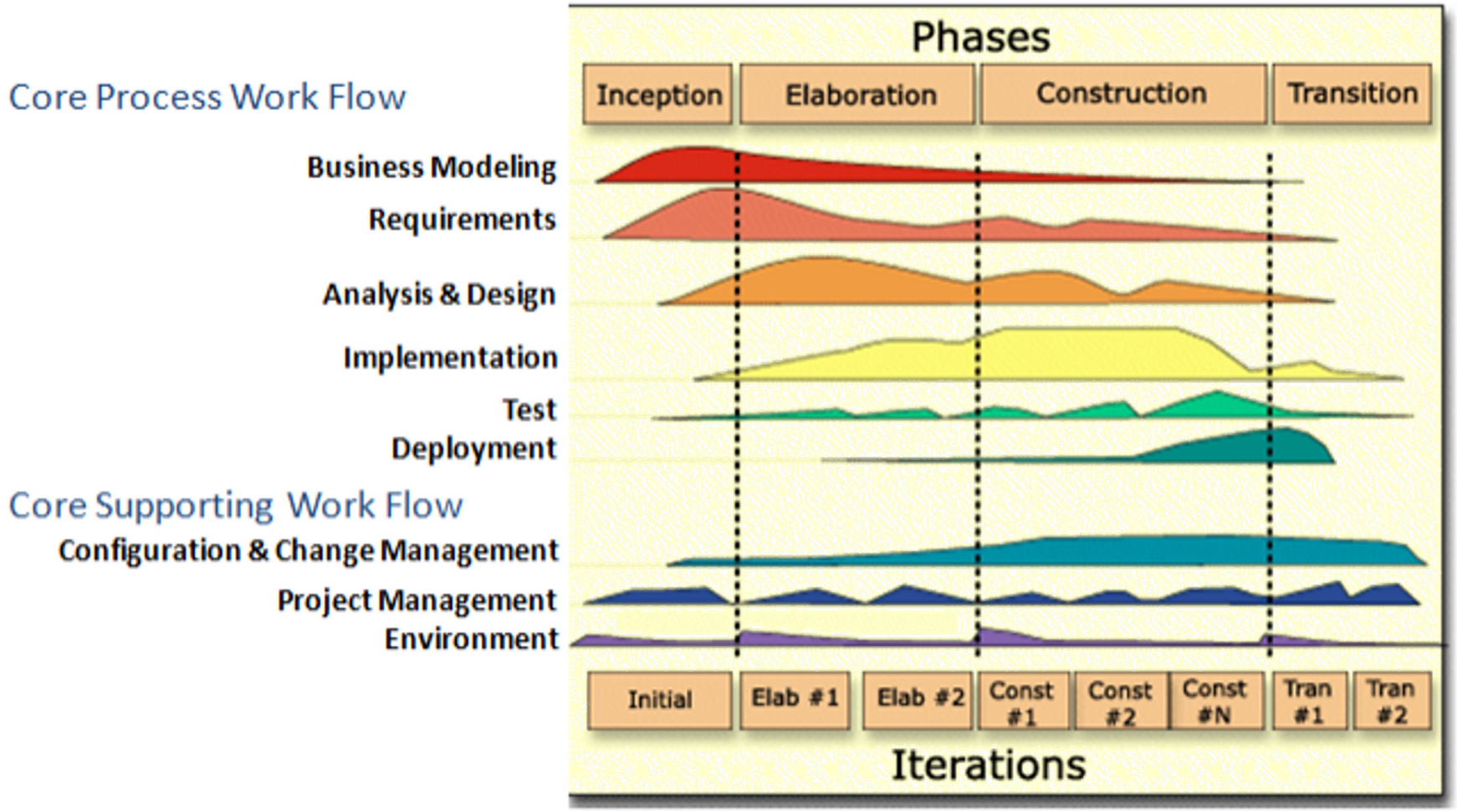
RUP

- *Rational Unified Process*
- Ferramenta de desenvolvimento de *software* iterativo
- Não tenta definir um processo concreto, mas sim uma estrutura de processo adaptável para selecionar os elementos do processo que são apropriados para as necessidades do projecto.

FASES DO *RUP*



FASES DO *RUP*


Core Process Work Flow
Business Modeling
Requirements
Analysis & Design
Implementation
Test
Deployment
Core Supporting Work Flow
Configuration & Change Management
Project Management
Environment
Phases
Inception
Elaboration
Construction
Transition
Initial
Elab #1
Elab #2
Const #1
Const #2
Const #N
Tran #1
Tran #2
Iterations

FASES DO *RUP*

- **Início / Concepção**
  - Delimitação do âmbito do projecto e estudo de viabilidade.
  - Estimativa de custos e cronograma do projecto.
  - Identificação de potenciais riscos.
  - Identificação dos principais requisitos (casos de uso críticos) e compreensão do sistema de forma abrangente (alinhamento com estratégia do negócio) .
- **Elaboração**
  - Incorpora a maior parte da análise de requisitos, a análise de domínio e o projeto.
  - Descrição da arquitectura de *SW* para todo o sistema, tendo em consideração os seus requisitos.
- **Construção**
  - Implementação e testes
- **Transição**
  - Instalação do Sistema
  - Disponibilização para os utilizadores
  - Formação
  - Avaliação

SCRUM

- Metodologia ágil para gestão dinâmica de projetos complexos
- Permite controlar de forma eficaz e eficiente o projeto, potencializando as equipas de trabalho
- Além de facilitar a definição de objetivos, também ajuda a cumprir os prazos estabelecidos

DESENVOLVIMENTO ÁGIL

- Forma de desenvolver recorrendo um *feedback* contínuo dos intervenientes
- Para desenvolver código funcional com alta qualidade
- Através de uma série de iterações curtas e bem delimitadas

INTERVENIENTES



Intervenientes

Product Owner

- Dono do produto ou projeto que vai ser trabalhado
- Representa os utilizadores ou clientes
- Entende do negócio
- Define quais os requisitos que vão fazer parte do *product backlog* e respetivas prioridades

INTERVENIENTES

SCRUM MASTER

- Faz a ligação entre o *product owner* e a equipa
- Faz com que o Scrum seja seguido
- Organiza os requisitos do *product backlog*
- Define as tarefas das *sprints*
- Orienta as reuniões
- Faz o acompanhamento do trabalho
- Certifica-se que cada integrante da equipa tem as ferramentas necessárias para cumprir a sua função da melhor forma possível
- Remove impedimentos e protege a equipa de riscos e interferências

Intervenientes

Dev Team

- Equipa que trabalha para o desenvolvimento do projeto ou produto
- Responsável pelas entregas
- Multifuncional, auto-organizada e autogerida

SCRUM


INPUTS FROM CUSTOMERS.
TEAM, MANAGERS & EXECS.
PRODUCT OWNER
THE TEAM
SPRINT MASTER
DAILY STAND UP MEETING
SPRINT REVIEW
FINISHED WORK
1-4 WEEK
SPRINT
Sprint end date and team deliverable do not change
PRODUCT BACKLOG
Prioritized a list of what is required:
Features , Bugs
SPRINT BACKLOG
TASK BREAKOUT
SPRINT RETROSPECTIVE

PRUCTO BACKLOG

- Registo que contém as funcionalidades do produto que devem ser desenvolvidas
- As funcionalidades devem estar ordenadas de acordo com a prioridade (das mais importantes para as menos importantes)
- Do *product backlog* é criado o *release backlog*
- O *release backlog* é um conjunto de tarefas para cada *sprint*

SPRINT

- Representa o espaço de tempo em que uma tarefa vai ser concluída
- O tamanho de cada *sprint* é adequado aos projetos e ao *release backlog*
- Cada *sprint* pode demorar entre uma a quatro semanas

Sprint Planning

- Reunião organizada pelo Scrum *Master* com o *PO* e a *Team*
- É definida a meta da *sprint* de acordo com as necessidades do *PO*, para um conjunto de funcionalidades
- São definidas as tarefas da *sprint*

Daily Scrum Meeting

- Reunião organizada pelo Scrum *Master* com a *Team*
- Dura cerca de 15 minutos
- É uma forma de comprovar que cada elemento está a cumprir o seu papel

Sprint Review

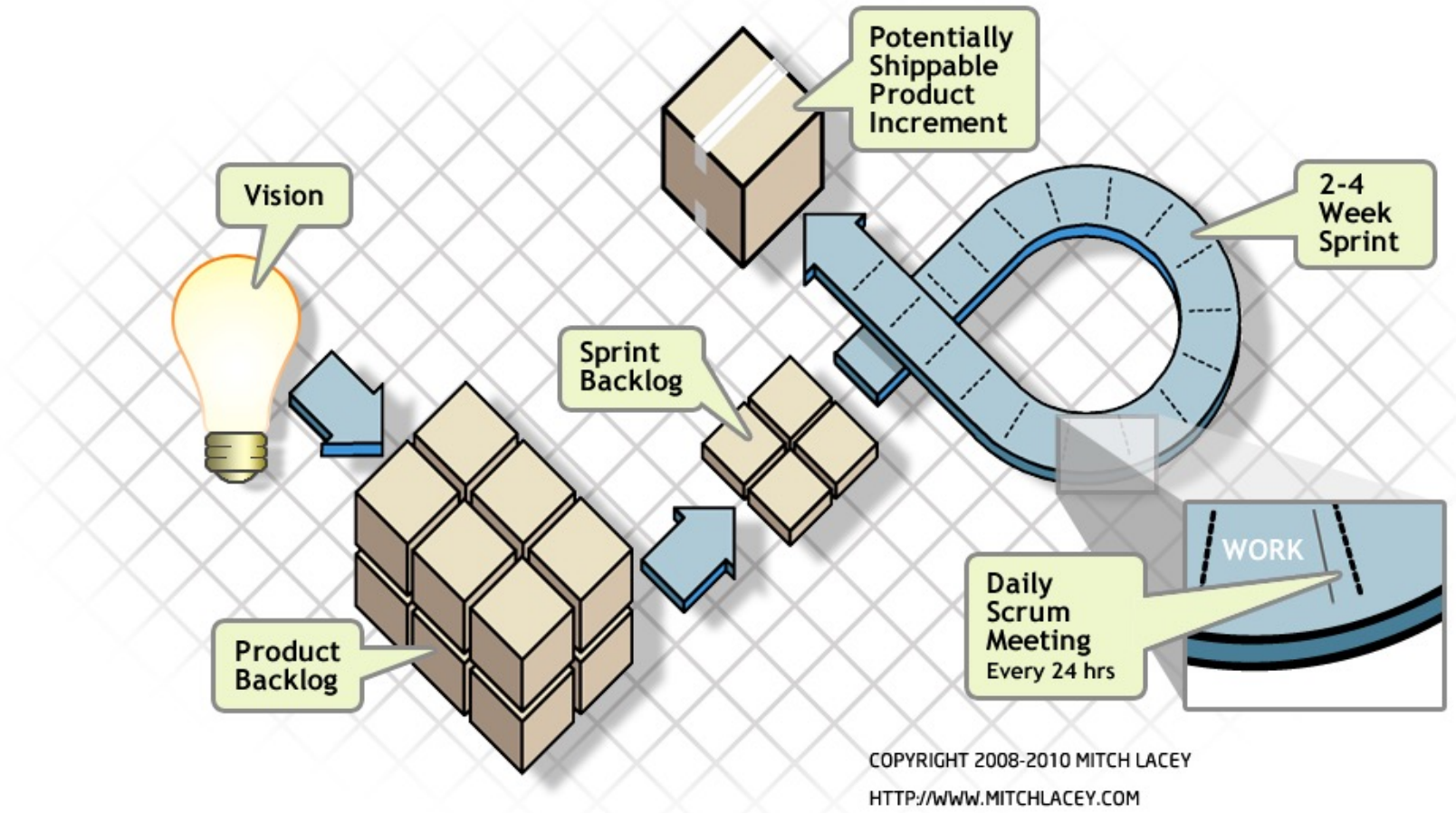
- A *Team* faz uma apresentação intermédia, ao *PO*, dos resultados conseguidos até ao momento

Sprint Retrospective

- Discussão do que foi conseguido com sucesso e do que não está de acordo com as especificações do cliente

- Definição de estratégias para adaptar ou reformular

SCRUM


Potentially Shippable Product Increment
Vision
2-4 Week Sprint
Sprint Backlog
WORK
Daily Scrum Meeting
Every 24 hrs
Product Backlog
COPYRIGHT 2008-2010 MITCH LACEY
HTTP://WWW.MITCHLACEY.COM

OBJETIVOS

- Metodologias de Desenvolvimento
- Engenharia de Requisitos
- User Stories
- Diagrama de Casos de Uso
- BPMN

LEVANTAMENTO DE REQUISITOS

- Técnicas para garantir que os objetivos do sistema sejam cumpridos, de forma consistente.
- Conjunto estruturado de objetivos/atividades usadas para derivar, validar e manter o projecto.
- Incluem o levantamento dos objetivos/*briefing*, análise, negociação e validação.

Levantamento de Requisitos

Instituto Superior de Tecnologias Avançadas do Porto

- Leitura de documentos
- Observação
- Entrevistas
- Reuniões
- Questionários
- Engenharia Reversa (reengenharia) das aplicações existentes

Levantamento de Requisitos

- Requisitos Funcionais
  - O que o sistema deve fazer

- Requisitos Não Funcionais
  - Atributos de qualidade do software
  - Como o sistema se deve comportar em relação à qualidade, desempenho, segurança e usabilidade, como: velocidade de resposta do sistema, a segurança dos dados, a escalabilidade, a facilidade de uso e os requisitos de conformidade com regulamentações.

EXEMPLO

- Supondo um briefing com um cliente que pretende a criação de uma aplicação *mobile* de gestão de *tickets* de atendimento num Centro de Saúde.

EXEMPLO

- Levantamento de requisitos **funcionais**:
  - Cada utilizador deve poder fazer registo, *login* e *logout*.
  - É permitido recuperar a *password*.
  - O utilizador pode fazer o pedido de um *ticket* para três tipos de atendimento: consulta marcada, urgência, enfermagem.
  - É possível verificar, para cada serviço, a que horas iniciou o atendimento, em que lugar vai, quantos faltam para o seu, média de tempo para atendimento do utente seguinte e data de término do respetivo atendimento.

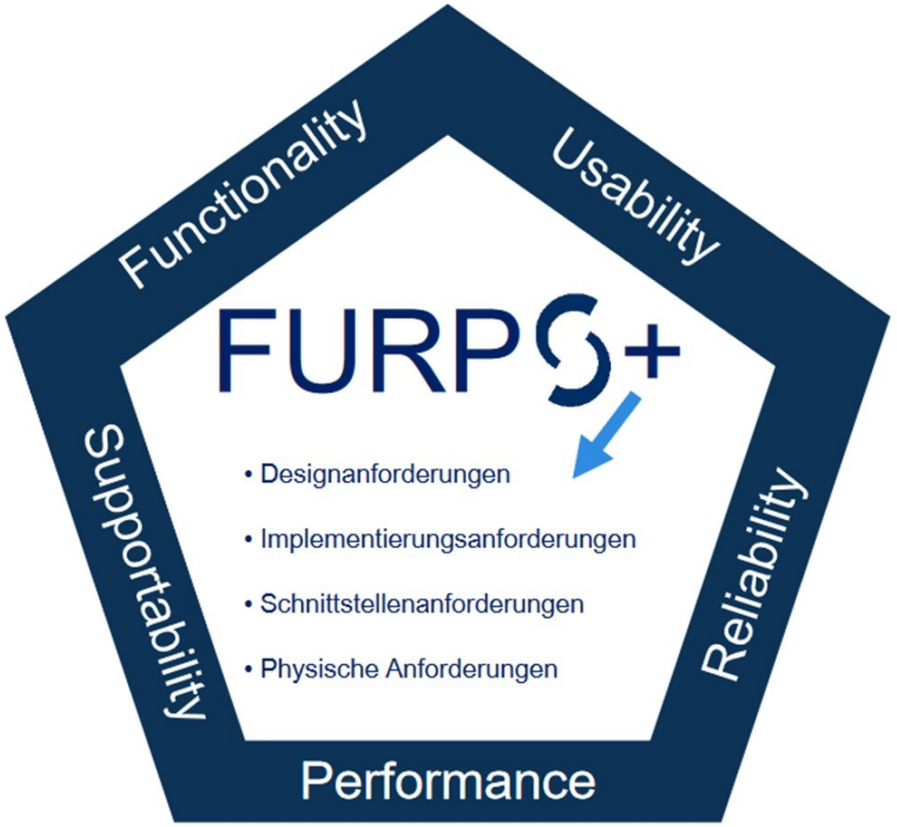
EXEMPLO

- Levantamento de requisitos **não funcionais**:
  - No caso de *ticket* para consulta marcada, a aplicação acede à base de dados e (de acordo com o Número do Utente) verifica se tem consulta, apenas gerando um *ticket* se tiver consulta marcada.
  - No caso de urgência gera, mas quando ultrapassa o limite de consultas de urgência estabelecido, informa que pode não ser possível o atendimento, se ultrapassar o limite final, o *ticket* não é gerado.
  - No caso de atendimento de enfermagem segue as mesmas regras que a urgência.
  - O sistema de *tickets* reinicia todos os dias às 23:59. A partir das 00:00 já é possível fazer um pedido de *ticket* para as consultas desse dia.
  - É possível pedir mais do que um *ticket*, mas apenas um para cada tipo de atendimento.
  - Se um utilizador fizer uso abusivo da aplicação será banido durante um determinado período de tempo.

EXEMPLO

- Levantamento de requisitos **não funcionais** (de qualidade):
  - A Base de Dados deve ser desenvolvida em MySQL.
  - A aplicação deve seguir uma arquitetura em 3 camadas (*UI*, *BLL* e *DAL*).
  - Deve usar-se *POO*.
  - A aplicação deve permitir guardar apenas uma foto por utilizador.

FURPS+


Functionality
Usability
Reliability
Performance
Supportability
FURPS+
• Designanforderungen
• Implementierungsanforderungen
• Schnittstellenanforderungen
• Physische Anforderungen

FURPS+

- **Functionality (Funcionalidade)** – representa todos os aspectos funcionais do software.
- **Usability (Usabilidade)** – é o atributo que avalia a interface com o utilizador. Incui: prevenção de erros; estética e design; ajudas (Help) e documentação; consistência e padrões.
- **Reliability (Confiabilidade)** – refere-se a integridade, conformidade e interoperabilidade do software. Inclui: falha; possibilidade de recuperação; possibilidade de previsão; exatidão; tempo médio entre falhas.
- **Performance (Desempenho)** – avalia os requisitos de desempenho do software. Inclui: tempo de resposta, consumo de memória, utilização da CPU, capacidade de carga e disponibilidade da aplicação.
- **Supportability (Suportabilidade)** – adaptabilidade, manutenibilidade, compatibilidade, configurabilidade, instalabilidade, escalabilidade, localizabilidade entre outros.

FURPS+

- O “+” do acrônimo engloba outros requisitos não-funcionais que devem ser lembrados:
  - Requisitos de design (desenho) – linguagens de programação, processo de software, uso de ferramentas de desenvolvimento, biblioteca de classes, etc.
  - Requisitos de implementação – padrões obrigatórios; linguagens de implementação; políticas de integridade de bases de dados; limites de recursos; e sistemas operativos.
  - Requisitos de interface – interface do sistema com o utilizador.
  - Requisitos físicos – especifica uma limitação física pelo hardware utilizado, por exemplo: material, forma, tamanho ou peso. Podendo representar requisitos de hardware, como as configurações físicas de rede obrigatórias.

OBJETIVOS

- Metodologias de Desenvolvimento
- Engenharia de Requisitos
- User Stories
- Diagrama de Casos de Uso
- BPMN

User Stories
Enunciado Exemplo

- Biblioteca de livros de histórias infantis em inglês “Storytail, Tales & Tails – English Books for Young Learners”
  - O objetivo é proporcionar a crianças, jovens e aos seus pais a possibilidade de aceder a uma plataforma web para imergir na língua inglesa, envolvida no mundo da fantasia, pela i) leitura; ii) audição e visualização de vídeos; e iii) exploração de atividades lúdicas e educativas, adaptadas à mensagem de cada livro.
  - Assim, a plataforma web permitirá que a cada utilizador seja possível i) pesquisar livros; ii) aceder ao conteúdo de cada livro; iii) explorar atividades; iv) marcar livros como favoritos/lidos; e v) outras funcionalidades descritas à frente.

User Stories
Exemplo

Como gestor de Bases de dados, eu pretendo perceber a relação da informação necessária ao sistema através do Modelo Relacional de Dados.

Como Gestor de Projeto, eu pretendo que a equipa distribua as *US* para cada elemento de forma justa num diagrama de Gantt.

Como Designer UI/UX, eu pretendo criar um protótipo da plataforma web com os vários ecrãs-chave, respeitando o manual fornecido "Brand Guidlines".

Como utilizador, eu pretendo aplicar filtros para facilitar a pesquisa de livros.

Como utilizador *logado*, eu pretendo aceder e alterar os meus dados pessoais.

Como administrador da plataforma, eu pretendo adicionar e editar livros no sistema.

Como *QA*, eu pretendo fazer testes funcionais da plataforma e documentar sucessos, insucessos e como foram resolvidos os insucessos.

Como Gestor de Projeto, eu pretendo preparar a *sprint review* para apresentação (*PPT* ou outro) no dia de avaliação (*deadline* da *sprint*)

User Stories
Exemplo

1

Como gestor de Bases de dados, eu pretendo perceber a relação da informação necessária ao sistema através do Modelo Relacional de Dados.

1

Como Gestor de Projeto, eu pretendo que a equipa distribua as *US* para cada elemento de forma justa num diagrama de Gantt.

2

Como Designer UI/UX, eu pretendo criar um protótipo da plataforma web com os vários ecrãs-chave, respeitando o manual fornecido "Brand Guidlines".

1

Como utilizador, eu pretendo aplicar filtros para facilitar a pesquisa de livros.

5

Como utilizador *logado*, eu pretendo aceder e alterar os meus dados pessoais.

1

Como administrador da plataforma, eu pretendo adicionar e editar livros no sistema.

3

Como *QA*, eu pretendo fazer testes funcionais da plataforma e documentar sucessos, insucessos e como foram resolvidos os insucessos.

4

Como Gestor de Projeto, eu pretendo preparar a *sprint review* para apresentação (*PPT* ou outro) no dia de avaliação (*deadline* da *sprint*)

OBJETIVOS

- Metodologias de Desenvolvimento
- Engenharia de Requisitos
- User Stories
- Diagrama de Casos de Uso
- BPMN

DIAGRAMA DE *USE CASES*

- Sequência de acções que um ou mais **actores** realizam num sistema para produzir um resultado observável com valor para esses actores.

- Conjunto de funcionalidades do sistema vistas pelos utilizadores, **sem considerar o comportamento interno do sistema**.

DIAGRAMA DE *USE CASES* – EXEMPLO

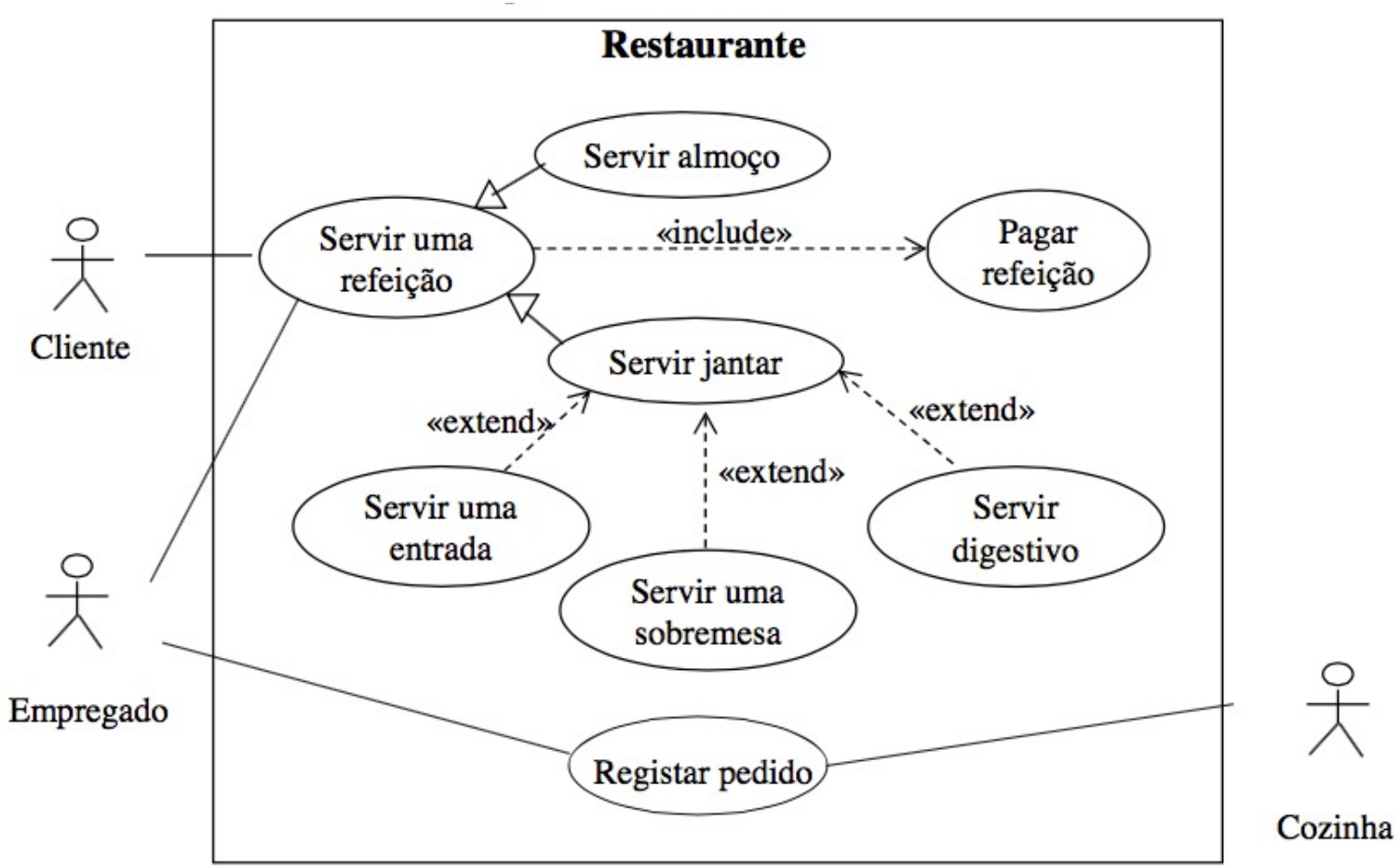


DIAGRAMA DE *USE CASES* – EXEMPLO

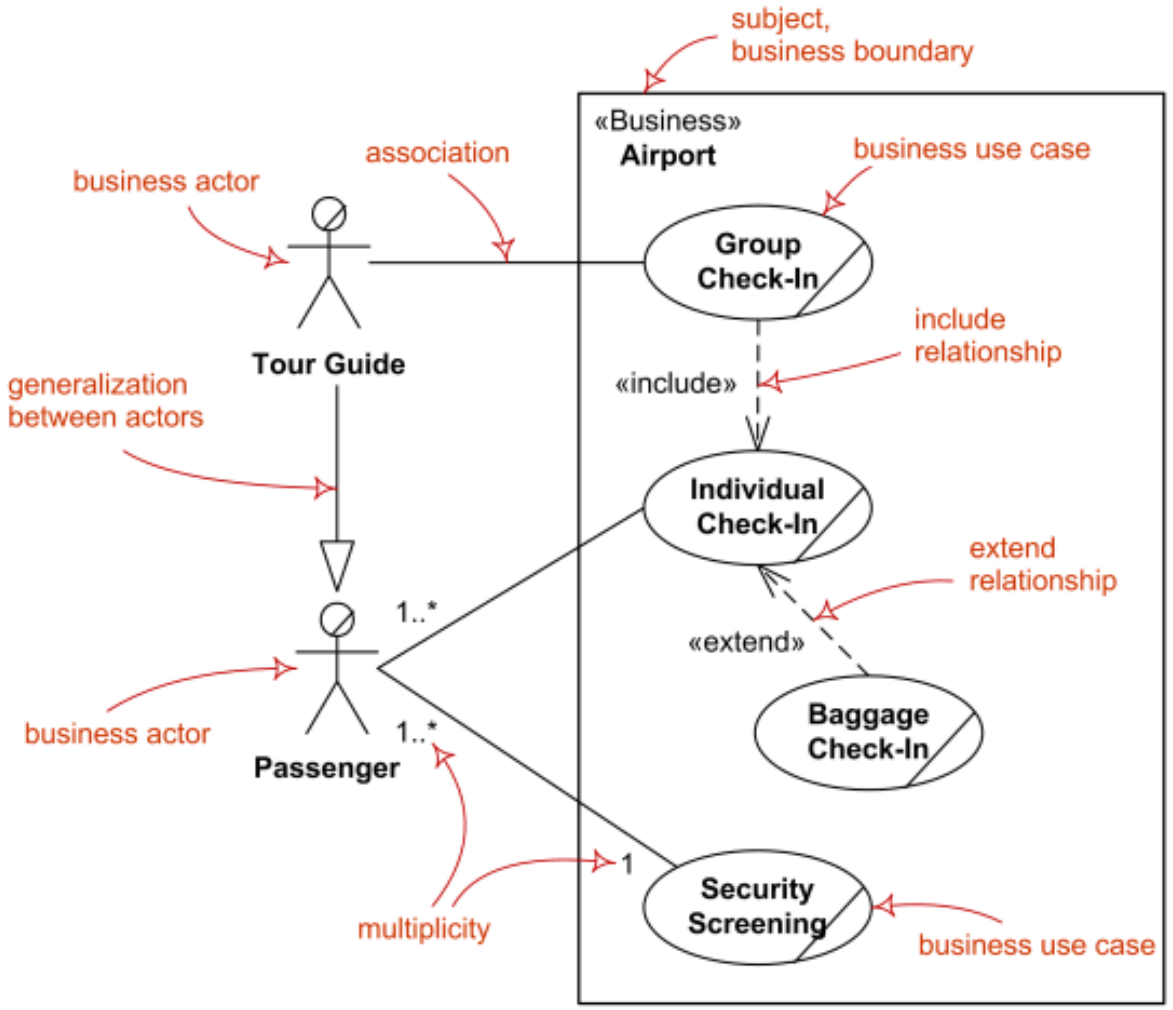
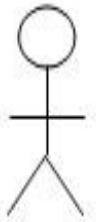


DIAGRAMA DE *USE CASES*

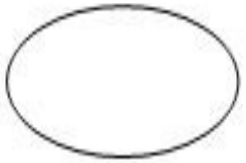
- Permite:
  - Especificar o **contexto** do sistema com quem interage (**actores**) e com que finalidade (**casos de utilização**)
    - Casos de Uso são funcionalidades do sistema vistas pelos utilizadores (**o que** faz e não **como** faz)
  - Modelar os **requisitos funcionais** do sistema
  - Pode referir-se a um sistema de software, um sistema de negócio ou organização, um equipamento, uma classe, etc.
  - É elaborado por analistas e especialistas de domínio nas fases iniciais do desenvolvimento (análise) e refinado nas fases seguintes (projecto) numa perspectiva de desenvolvimento iterativo e incremental.

SIMBOLOGIA

- Actor



- *Use Case*

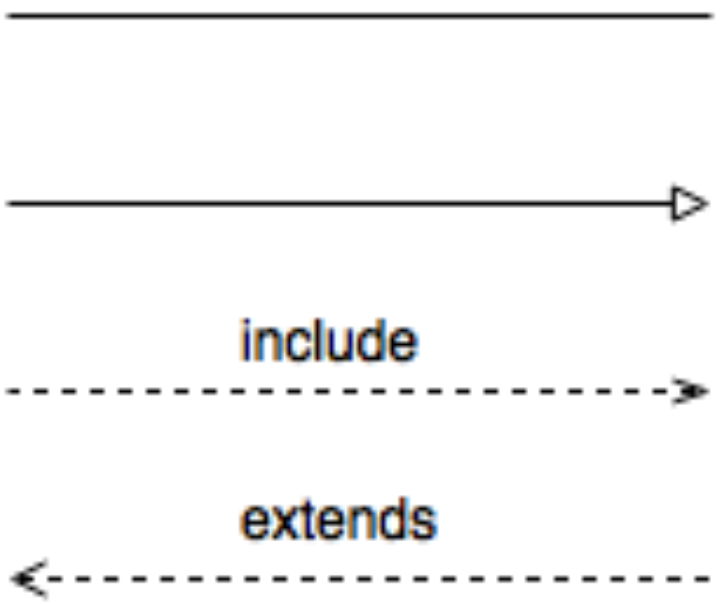


- Sistema



SIMBOLOGIA

- Associação
- Generalização/Herança
- *Include*
- *Extends*



ACTOR

- É um papel que alguma entidade do ambiente envolvente ao sistema desempenha quando interage com o sistema.

GENERALIZAÇÃO/HERANÇA

- Usufrui das mesmas propriedades que uma relação pai/filho, onde o *use case* filho herda o comportamento do pai

GENERALIZAÇÃO/HERANÇA – EXEMPLO

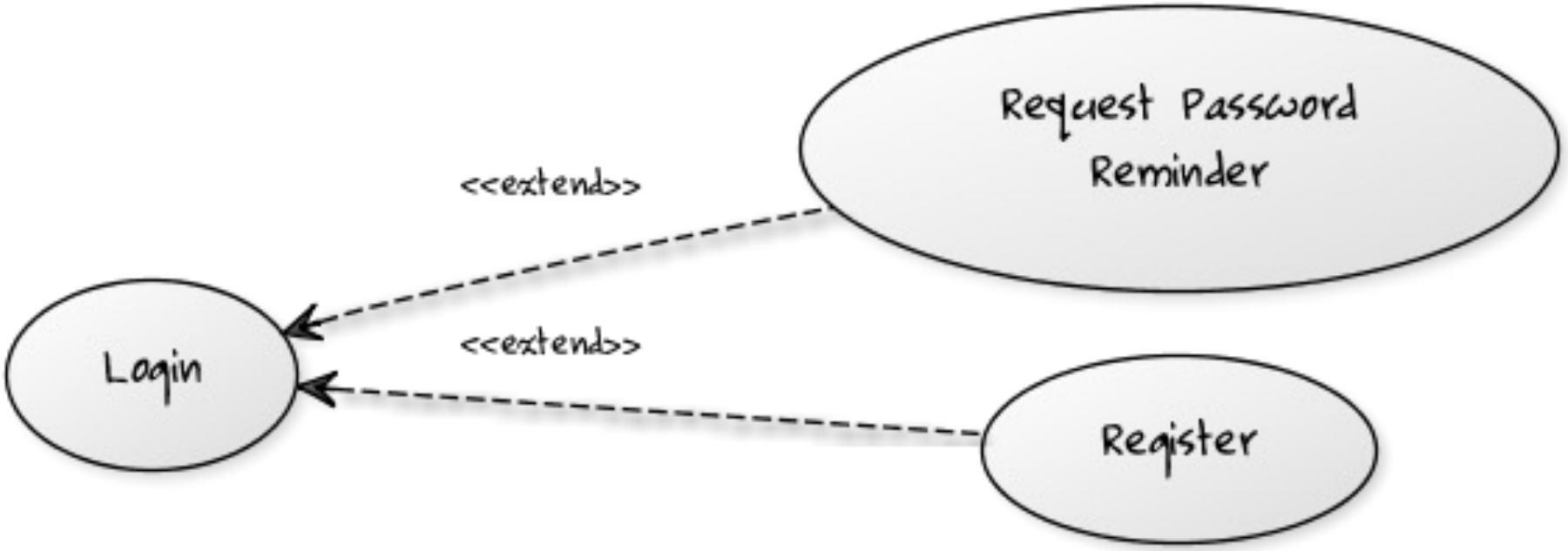


Inclusão vs Extensão

- **Inclusão** <<include>> (seta a tracejado)
  - O caso base incorpora o comportamento do outro
  - Corresponde a uma relação de delegação
  - Evita repetir os mesmos fluxos de acção

- **Extensão** <<extend>> (seta a tracejado)
  - O caso filho pode ser estendido com o comportamento de outro(s).
  - Permite representar subfluxos de acções alternativos

Inclusão vs Extensão – Exemplo



TEMPO

- Parte-se do princípio que todos os *Use Cases* são originados por actores, contudo existem alguns que podem ser despoletados pelo sistema

- Exemplo:
  - Cópia de segurança para uma unidade de backup
  - Enviar promoções mensais a uma lista de clientes

Descrição Estruturada de um *Use Case*

- Também é comum encontrem-se as funcionalidades de um sistema descritas em texto livre para complementar os diagramas gráficos.

Pré Condição

- Pré-condição
  - Indica o que deve existir inicialmente para que o cenário descrito seja seguido com sucesso

Cenário Principal e Secundário

- Cenário Principal
  - Onde não surgem problemas
- Cenário Secundário
  - Onde são descritos caminhos alternativos, quando se pensa no que poderá correr mal

Pós Condição

- Pós-condição
  - É demonstrado o que irá acontecer depois do cenário ser concluído

EXEMPLO

- Efectuar uma encomenda pela internet:
  - O cliente, após ter validado o seu acesso, pesquisa produtos, sendo mostrado o catálogo dos produtos. Para adicionar um produto terá apenas de seleccionar o mesmo para que os seus dados sejam visualizados. Ao mesmo tempo é calculado o valor da encomenda.
  - Através da confirmação do cliente, o sistema passa para a função de pagamento.
  - Depois da introdução e confirmação desses dados é atribuído um código à encomenda, que posteriormente será entregue na morada indicada.

EXEMPLO (RESOLUÇÃO)

- Use Case
  - Efectuar uma encomenda pela internet:
- Pré-condição
  - O cliente é um utilizador registado no sistema
- Descrição (Cenário Principal)

1. O cliente seleciona a opção de visualizar produtos
2. O sistema mostra o catálogo de produtos
3. O cliente adiciona produtos à encomenda
4. O sistema mostra informação do produto
5. O sistema (re)calcula o valor da encomenda
6. O cliente confirma a encomenda
7. O sistema pede os dados de pagamento
8. O sistema confirma os dados do pagamento e atribui código à encomenda

- Pós-condição
  - A encomenda será registada na base de dados

EXEMPLO (RESOLUÇÃO)

- Caminhos Alternativos (Cenário Secundário)

3.1 Se o código que o cliente escolheu, pertencer a um produto sem stock, o sistema deve avisar com uma mensagem

8.1 Se os dados de pagamento forem inválidos, o sistema informa o cliente e volta para o ponto 7.

EXEMPLO (RESOLUÇÃO)

- Use Case
  - Efectuar uma encomenda pela internet:
- Pré-condição
  - O cliente é um utilizador registado no sistema
- Descrição (Cenário Principal)

1. O cliente seleciona a opção de visualizer produtos
2. É mostrado o catálogo de produtos
3. O cliente adiciona produtos à encomenda
4. O sistema mostra informação do produto
5. O sistema (re)calcula o valor da encomenda

   5.1 Extend: Calcular Desconto
6. O cliente confirma a encomenda
7. O sistema pede os dados de pagamento
8. O sistema confirma os dados do pagamento e atribui código à encomenda

- Pós-condição
  - A encomenda será registada na base de dados

EXEMPLO (RESOLUÇÃO)

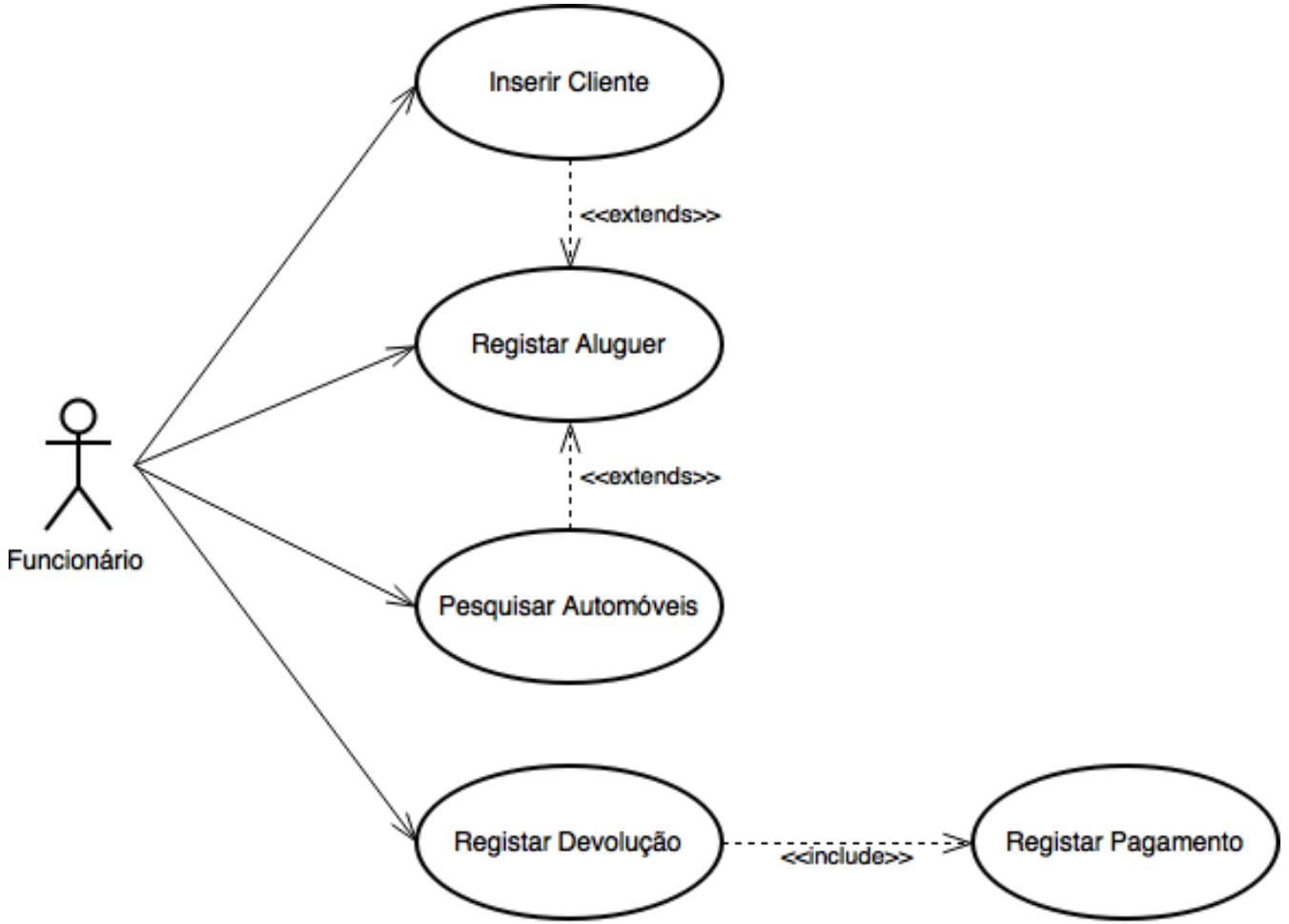
- Use Case
  - Calcular Desconto
- Pré-condição
  - O produto está com desconto internet
- Descrição (Cenário Principal)

1. O sistema retorna o valor do desconto
2. Mostra o desconto na encomenda
3. Calcula o total subtraindo o preço do desconto

PROBLEMA 1

- Pretende-se desenvolver um software de suporte a diversas actividades de uma empresa de aluguer de automóveis.
- O software é apenas utilizado pelos funcionários em loja.
- Este software deve permitir registar contratos de aluguer, entregas e devoluções de automóveis, com cálculo automático do preço a pagar pelos seus clientes.

DIAGRAMA DE *USE CASES* – EXEMPLO



OBJETIVOS

- Metodologias de Desenvolvimento
- Engenharia de Requisitos
- User Stories
- Diagrama de Casos de Uso
- BPMN

BPMN

- *Business Process Modeling Notation*
- Usados para comunicar visualmente através de diagramas.
- Têm como objetivo principal criar um padrão numa linguagem comum para modelação de processos de negócios.
- Esta notação foi desenvolvida para melhorar a comunicação entre departamentos e pessoas, uma vez que mostra um processo dividido por fases. Assim, através de símbolos universais, um profissional técnico, analista, programador, ou até de qualquer profissional de uma dada área específica, interpreta um problema sem questões dúbias e de forma otimizada.
- **“Só podemos mudar o que entendemos, e só entendemos o que podemos visualizar.”**

PORQUÊ BPMN?

- Fácil de usar e de perceber
- Orientado para o cliente
- Ferramenta independente
- Standard na indústria

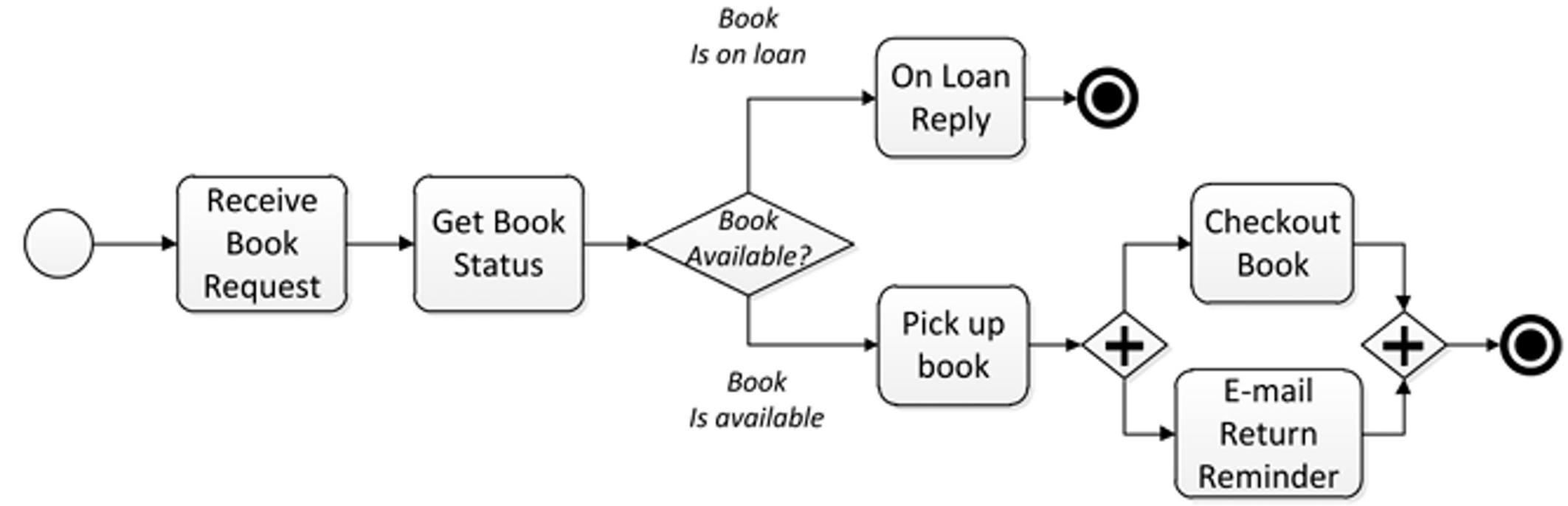
SIMBOLOGIA

- **Área**: simboliza os papeis e responsabilidades no processo;
- **Eventos:** indicam ocorrências exteriores ao processo que o influenciam;
- **Atividades:** representam tarefas a serem realizadas;
- **Gateways:** mostram a ramificação e a reunião do fluxo de tarefas e determinam o caminho que o processo seguirá;
- **Artefactos:** documentação externa;
- **Conectores:** elementos de ligação da sequência dos fluxos de trabalho.

SIMBOLOGIA

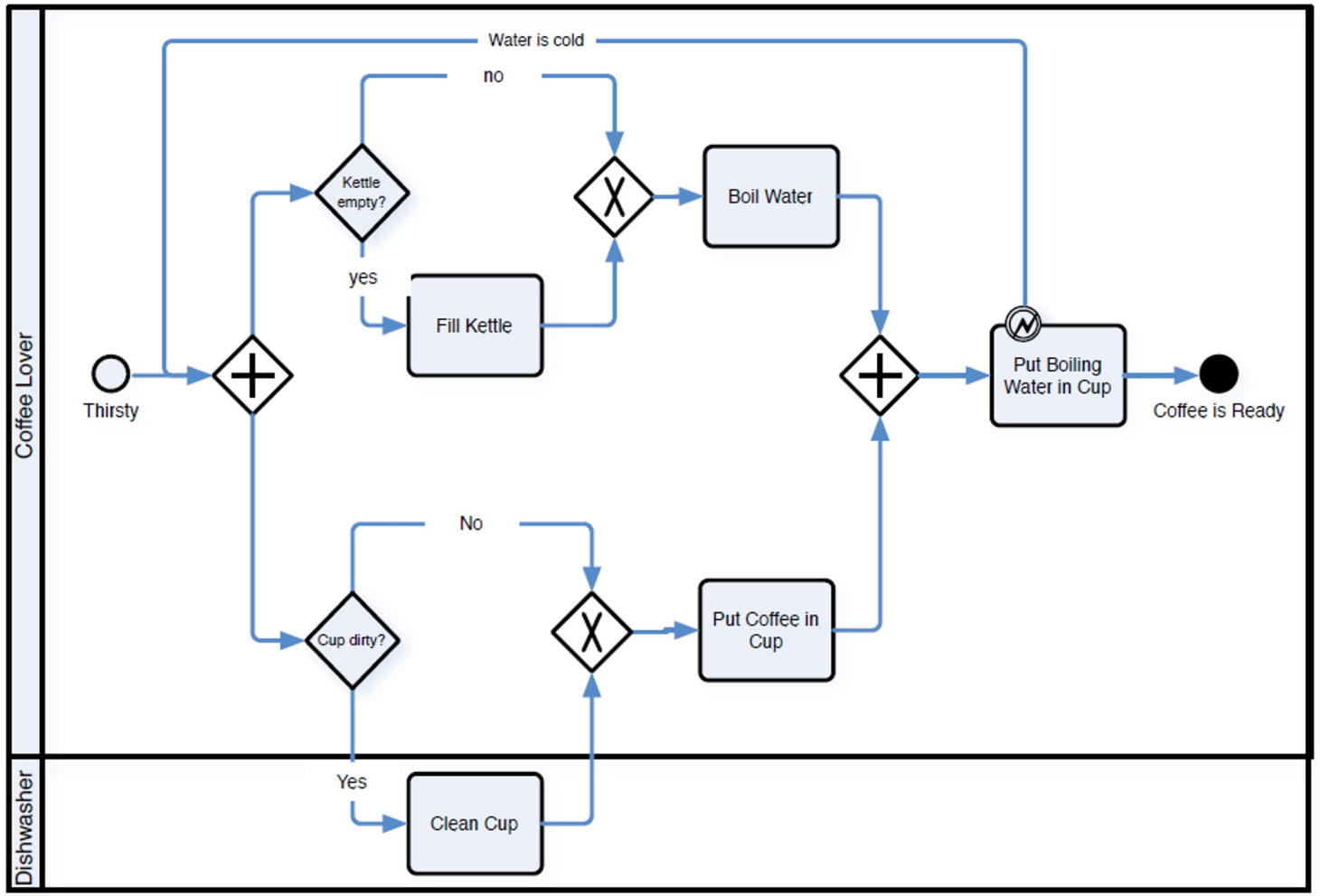

Conectores
Fluxo de sequência das atividades
Fluxo de mensagens
Associação de artefatos e elementos do fluxo
Gateways
Exclusivo: o fluxo segue por uma condição exclusiva
Paralelo: o fluxo se divide em outros que ocorrem em paralelo
Inclusivo: o fluxo segue por uma condição inclusiva, para cada fluxo de sequencia é avaliada uma fórmula, se for retornado o valor verdadeiro o caminho é usado.
Complexo: Controla condições complexas de divergência e também convergência.
Inicial exclusivo (eventos): divide o fluxo iniciando um processo ao ocorrer um evento exclusivo.
Intermediário exclusivo (eventos): usado para dividir o fluxo iniciando um processo ao ocorrer um evento exclusivo.
Atividades
Tarefa
Tarefa simples
Tarefa de serviço
Serviço
Enviar mensagem
Tarefa de envio de mensagem
Tarefa de recebimento de mensagem
Mensagem
Manual
Tarefa manual
Tarefa de usuário
Usuário
Script
Tarefa de script
Tarefa de regra de negócio
Regra de Negócio
Eventos
Eventos Iniciais
Início de processo simples
O processo é iniciado com a chegada de uma mensagem
Tempo: o processo é iniciado por uma condição de tempo
Condicional: uma condição lógica determina o início do evento
Sinal: um sinal vindo de outro processo inicia este processo
Múltiplo: um de muitos eventos possíveis iniciam o processo.
Múltiplo paralelo: múltiplos eventos devem ocorrer para iniciar o processo.
Eventos Intermediários
Eventos Final
Escalação: uma exceção de negócio inicia o processo
Link: um link é acessado durante o processo
Final

BPMN
Exemplo Simples



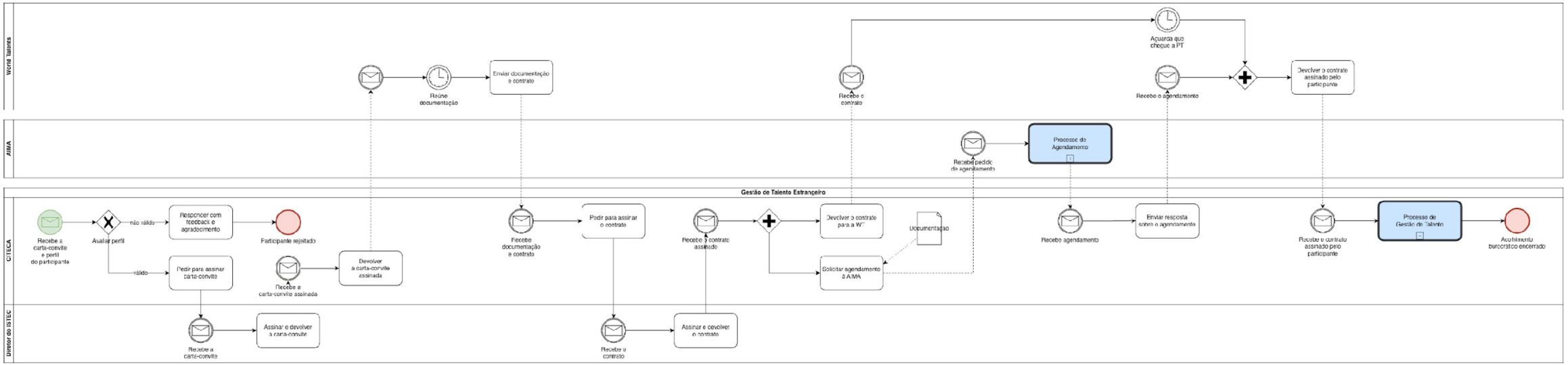
BPMN

Exemplo com Lanes



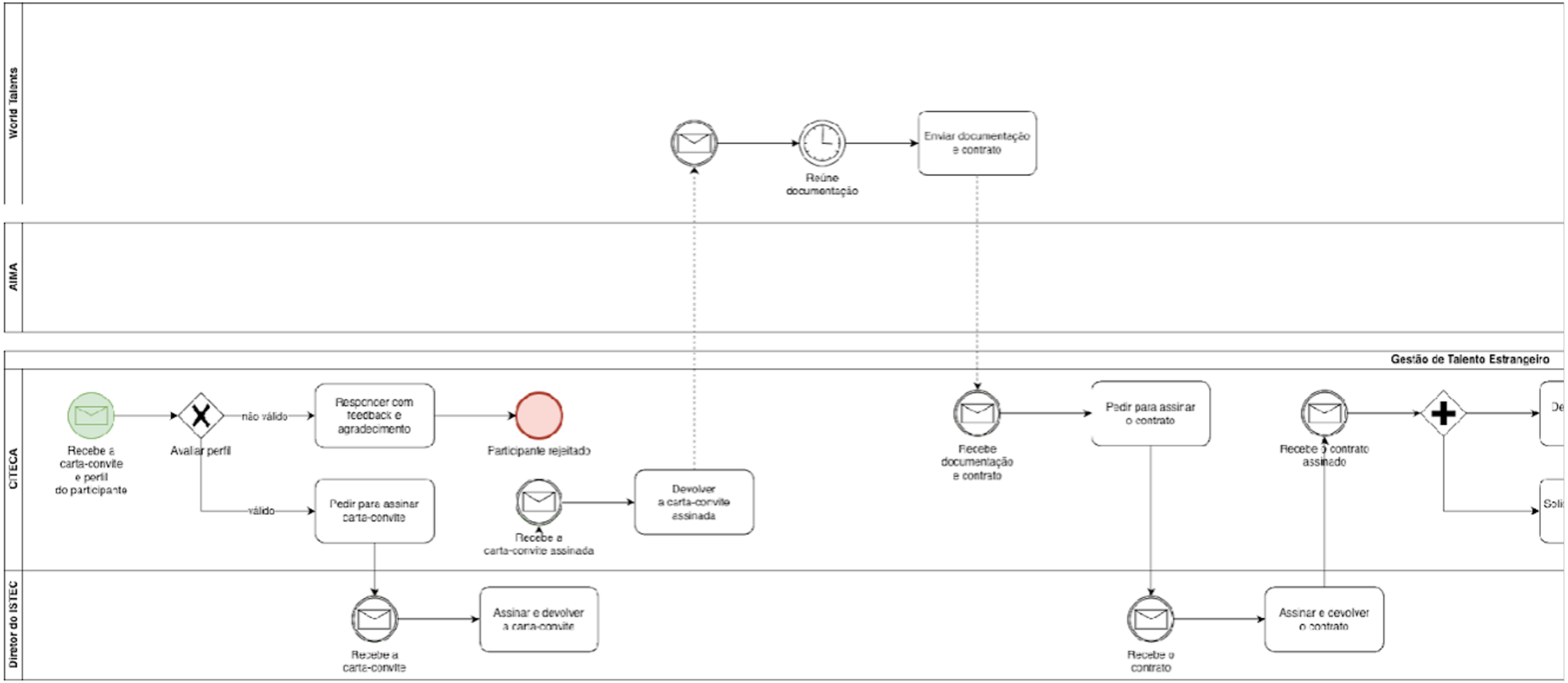
BPMN

Exemplo com Lanes



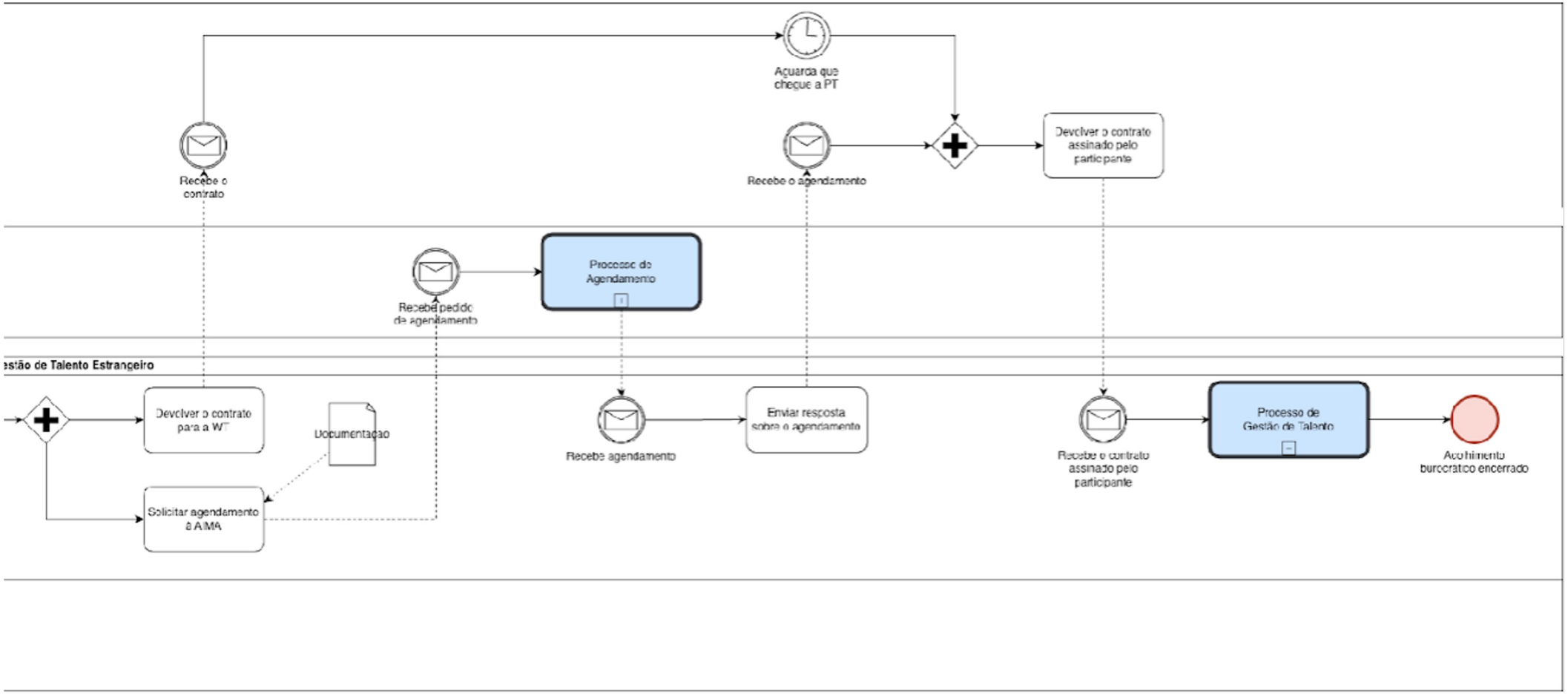
BPMN

Exemplo com Lanes



BPMN

Exemplo com Lanes



BPMN
Boas Práticas

- O fluxo de sequência nunca cruza fronteiras de Pools.
- Todo o processo deve ter início e fim definidos (mesmo que implícitos, mas explicitar é melhor).
- Os gateways não executam trabalho, apenas encaminham.
- Os fluxos que divergem devem, eventualmente, convergir ou terminar.
- Modelar o fluxo da esquerda para a direita facilita a leitura temporal.
- Usar "Verbo + Substantivo" para tarefas (ex: "Validar o login de utilizador").
- Se um modelo tem mais de 30 elementos, considere usar Subprocessos.
- Não criar ciclos que nunca satisfazem a condição de saída.
- Não criar atividades que recebem fluxo mas não geram saída.
- As atividades geram uma e apenas uma saída.

Helder Rodrigo Pinto
helder.pinto@my.istec.pt

